

1. INTRODUCTION

In recent days the detection of malicious activity in a network is very important for data security and system integrity. Intrusion Detection Systems (IDS) plays a key role in this because they monitor the network traffic and alert potential threats in real-time. SNORT is an open-source IDS which uses rule-based detection methods to identify the known patterns of malicious behavior in the network so it is known as powerful network security tool. The process involves configuring the SNORT to detect common network anomalies such as ICMP echo requests, TCP SYN floods, SSH brute-force attacks, SQL injection attacks, and other types of malicious traffic. By setting up custom rules and generating test traffic, we can verify SNORT's capability to detect threats and analyze the consequences of detected anomalies. we better understand how SNORT contributes to improving the security of networks and protects against future cyber attacks.

2. SETUP PROCESS

2.1 DIRECTORY STRUCTURE CREATION

For the effective and consistent SNORT environment there is a need of a certain primary directories which are created in order to store configuration files, rules, logs, and plugins. These directories help in creating a clean separation of concerns so that easy configuring, monitoring, and maintenance is ensured for the system. For example, /etc/snort stores all primary configuration files and /var/log/snort is designated to store logs for inspection and also the directories like /etc/snort/rules and /usr/local/lib/snort_plugins provide efficient management of detection rules and optional plugins, respectively. It is an efficient way for an organization to make the SNORT operate without any issues or security threats.

2.1.1 MAIN CONFIGURATION DIRECTORY

Command: `sudo mkdir /etc/snort`

Explanation: This command is used to configure the main directory of SNORT configuration files. The /etc/snort is the file where all of the SNORT configuration meets the master configuration file snort.conf and other crucial configurations. It is the most superiorly positioned directory for all SNORT entities.

2.1.2 RULES DIRECTORY

Command: `sudo mkdir -p /etc/snort/rules`

Explanation: This command makes a subdirectory under /etc/snort where rules to be saved are custom and predefined. The /etc/snort/rules directory has an important role in processing detection rules that specify the type of traffic SNORT is supposed to detect and report. It gives room to include or modify rules easily without altering the original configuration.

2.1.3 LOG DIRECTORY

Command: `sudo mkdir -p /var/log/snort`

Explanation: This command creates a directory by itself to store log files generated by SNORT. The `/var/log/snort` directory provides a place where all alerts and logs of network actions are stored in an orderly fashion. These logs are the important which are used for the examination of events detected, evaluation of possible threats, and troubleshooting.

2.1.4 PLUGINS DIRECTORY

Command: `sudo mkdir -p /usr/local/lib/snort_plugins`

Explanation: This command sets up a directory to load additional plugins that introduce capabilities to SNORT. The `/usr/local/lib/snort_plugins` directory adds the ability to have third-party or custom-built plugins, possibly providing features such as extended protocol support or application-specific detection ability.

2.1.5 BUILT-IN RULES DIRECTORY

Command: `sudo mkdir -p /etc/snort/builtin_rules`

Explanation: This command creates a directory for management of built-in rules provided by SNORT. The `/etc/snort/builtin_rules` contains preconfigured rules shipped with SNORT to allow easy management of default detection tools. They address wide types of common network attacks.

2.1.6 SHARED OBJECT RULES DIRECTORY

Command: `sudo mkdir -p /etc/snort/so_rules`

Explanation: This command makes a directory for dynamically loaded rules that are run as shared object (SO) files. The `/etc/snort/so_rules` directory facilitates rule modularization, allowing administrators to load rules individually at runtime without modifying the core configuration. This enhances flexibility and scalability in rule deployment.

2.2 CONFIGURATION FILE (SNORT.CONF)

`snort.conf` is the main configuration file for SNORT where the network settings and rules are set. To customize the operation of the SNORT but the file was edited by the command.

Command: `sudo nano /etc/snort/snort.conf`

Explanation: It is used to edit the `snort.conf` file using the Nano text editor in order to alter the configuration.

Configurations: The `home_net` is set as `10.0.2.0/24` which sets the internal network that SNORT will be searching for suspicious activity on and `external_net` is set to any so all the traffic coming from the outside is examined for analysis. The `global_default_rule_state` is enabled which allows all the rules to be enabled by the

default to provide full coverage for detecting the potential threats without having to manually enable each of the rule. In addition to the configuration includes the local.rules file in which contains personal rules that can accommodate particular security requirements and making it possible for advanced and tailored detection capabilities. With this configuration the SNORT can adequately observe internal and external traffic and use default and individualized rules to detect and alert on malicious activity. By organizing the configuration in this way, SNORT is optimally placed to offer effective and efficient network protection.

2.3 CUSTOM RULES (LOCAL.RULES)

A custom rules file was created to define specific types of traffic that SNORT should be looking out for using the command:

Command: `sudo nano /etc/snort/rules/local.rules`

Explanation: Opens the local.rules file in edit mode to allow adding custom rules to detect specific network anomalies and potential threats.

Rules:

ICMP Test: Alerts on ICMP echo requests to the home network which indicates the general ping activity.

```
alert icmp any any -> $HOME_NET any (msg:"ICMP Test"; sid:1000001; rev:1;)
```

ICMP Ping Sweep: Creates an alert for multiple ICMP requests from the same source within one second which can be an indication of scanning activity.

```
alert icmp any any -> any any (msg:"ICMP Ping Sweep"; detection_filter:track by_src, count 5, seconds 1; sid:1000002; rev:1;)
```

TCP Port Scan: It checks for the potential TCP SYN floods by watching the multiple SYN packets from the same source which is one of the technique employed by port scanning attacks.

```
alert tcp any any -> any any (msg:"Potential TCP Port Scan"; flags:S; detection_filter:track by_src, count 5, seconds 2; sid:1000003; rev:1;)
```

SSH Brute Force: The notifications on the multiple connection attempts to port 22 may indicates the automated brute-force attacks against SSH services.

```
alert tcp any any -> any 22 (msg:"Potential SSH Brute Force"; flags:S; flow:stateless; detection_filter:track by_src, count 5, seconds 10; sid:1000004; rev:1;)
```

Suspicious User Agent: It alerts on HTTP requests with suspicious user-agent strings such as those belonging to scanning software or bots and that could signify malicious intent.

```
alert tcp any any -> any 80 (msg:"Suspicious User Agent - Possible Scanner";  
flow:established,to_server; content:"User-Agent|3A| "; http_header;  
pcre:"/(?::Scan|Proxy|Spider|Nmap)/i"; sid:1000005; rev:1;)
```

SQL Injection Attempt: It alerts on SQL injection attempts by searching for the %27 character in HTTP requests and a common indicator of the injection attempts.

```
alert tcp any any -> any any (msg:"SQL Injection Attempt";  
flow:to_server,established; content:"%27"; sid:1000006; rev:1;)
```

FTP Brute Force: It monitors several login attempts to FTP servers which may be a sign of brute-force attacks attempting to gain unauthorized access.

```
alert tcp any any -> any 21 (msg:"FTP Brute Force Attempt";  
flow:established,to_server; detection_filter:track by_src, count 5, seconds 10;  
sid:1000007; rev:1;)
```

DNS Zone Transfer Attempt: It detects an unauthorized DNS zone transfer requests which can disclose confidential DNS information and breach network security.

```
alert tcp any any -> $HOME_NET 53 (msg:"DNS Zone Transfer Attempt";  
flow:established,to_server; content:"|00 00 FC|"; offset:12; depth:3; sid:1000008;  
rev:1;)
```

These are the custom rules enable SNORT to detect a wide range of potential threats from basic ICMP traffic to advanced attacks like SQL injection and brute-force attacks. By Improving the rules to the specific needs of the network the SNORT provides enhanced visibility and protection against general and advanced cyber threats.

2.4 PERMISSIONS

To ensure SNORT operates securely and sensitive configuration files, logs, and plugins cannot be accessed by unauthorized individuals the file permissions were set using the commands:

Command: `sudo chmod 666 /var/log/snort`

Explanation: This command modifies the permissions of the /var/log/snort directory to 666 by giving the read and write permissions to all users. This permits the SNORT to write logs to this directory without any restriction but still permits administrators to read log files when needed.

Command: `sudo chmod -R 750 /etc/snort`

Explanation: This recursive switch (-R) assigns 750 permissions to the /etc/snort directory and all of its subdirectories and files. The 750 permission restricts access so that only the rootuser can read, write, and execute, group members can read and execute, and others cannot access. This secures the SNORT configuration files and cannot be modified by users who do not have the rights to do so.

Command: `sudo chmod -R 750 /var/log/snort`

Explanation: It is similar to the above command but it is assigning 750 permissions recursively to the /var/log/snort directory. Restricting access to the log files and this will prevent unwanted users from seeing or changing critical information collected by using the network monitoring.

Command: `sudo chmod -R 750 /usr/local/lib/snort_plugins`

Explanation: This command grants 750 permissions to the /usr/local/lib/snort_plugins directory and everything in it. This will protect the plugin files which can be the additional features or custom modules for SNORT from being changed or accessed without permission.

3. TESTING AND RUNNING SNORT

To verify the SNORT configuration and to begin the monitoring network traffic the following commands were used:

Command: `sudo snort -T -c /etc/snort/snort.conf -i eth0`

Explanation: This command tests the SNORT configuration file (/etc/snort/snort.conf) for syntax errors and verifies all the configurations are loaded as required without starting the service. The -T option performs a test of the configuration and the -c option specifies the location of the configuration file. In addition, the -i eth0 option specifies the test over the eth0 network interface. This is necessary to identify and correct any issue in the configuration before executing SNORT in live mode.

Command: `sudo snort -c /etc/snort/snort.conf -i eth0 -A alert_fast`

Explanation: This command runs SNORT in live mode to listen to live traffic on the eth0 network interface. The -c option specifies the configuration file which ensures that SNORT loads the given rules and settings. The -i eth0 option specifies the network interface to listen to and the -A alert_fast option causes SNORT to output alerts as alert_fast format which provides a quick summary of the detected events. This mode of operation allows SNORT to actively perceive and log potential threats as they emerge in the network.

4. ALERT LOGS

While executing SNORT, several alerts were raised from the custom rules defined in the local.rules file, which indicated probable security violations in the network. The system detected an ICMP Test alert when an ICMP echo request was sent from a remote source (192.168.1.10) to the internal network, indicating probable reconnaissance. And, a Ping Sweep via ICMP was identified as multiple ICMP requests from the same location within a couple of seconds, sweeping various hosts in the inner subnet, being one of the typical features of probing or mapping in the network. A TCP Port Scan alert came into force where multiple SYN packets were observed reaching multiple ports to one host (10.0.2.10) and showing fear of attack to discover accessible services. Moreover, a SSH Brute Force attack was noticed, with constant failed logins on port 22, demonstrating the likely attack aimed at unauthorized access to the SSH service. Lastly, a Suspicious User Agent alert was issued when an HTTP request with a user-agent string usual for scanning tools or bots was detected and identifying the likelihood

of malicious web traffic. These warnings altogether demonstrate the capability of SNORT in identifying and recording the potential threats which may be beneficial to proactive network security and incident response.

5. SECURITY IMPLICATIONS

More ICMP traffic may be a sign of DoS or reconnaissance attacks where attackers flood the network with excessive requests to over utilize the resources. TCP SYN floods are also common DoS/DDoS tactics because attackers send multiple connection requests to exhaust server resources which disrupts service. Brute-force activities on the SSH or FTP services mean repeated attempts to crack credentials automatically and this carries a high risk of unauthorized access. SQL injection attacks exploit vulnerabilities in web applications to allow attackers to manipulate the databases and steal the sensitive information. Lastly the unauthorized DNS zone transfers can disclose the sensitive DNS data which can be utilized by the attackers to map the network infrastructure for potential future exploitation and thereby compromising overall network security.

6. SECURITY ANALYSIS

SNORT settings effectively detected generic network threats by monitoring specific traffic and reducing false positives through directed rules and detection filters like (detection_filter:track by_src). SNORT is excellent at identifying known attack patterns but lags behind in detecting zero-day attacks or APT's. It is recommended to update the rules regularly and incorporate threat intelligence and incorporate SNORT into firewalls or IPS for a higher defense posture.

7. CONCLUSION

Configuring SNORT proves the necessity of securing networks from malicious operations. It does this through real-time monitoring of traffic and warning of its threats. Customized rules and log examination bring us useful knowledge of possible threats and security ramifications. Whereas SNORT is a good security mechanism for established attack pattern detection it should be augmented with other security modules such as firewalls, intrusion prevention systems (IPS), and threat feeds intelligence in order to take care of expanding cyber threats and secure the network. The system brings into perspective the employment of a multi-layered security approach for prevention against current as well as evolving threats.

8. REFERENCES

SNORT DOCUMENTATION: <https://www.snort.org/documentation>
TALOSINTELLIGENCE: <https://talosintelligence.com/>
SANS: <https://www.sans.org/>

9. APPENDIX

9.1 APPENDIX 1: COMMANDS USED

9.1.1 DIRECTORIES CREATION:

```
sudo mkdir /etc/snort
sudo mkdir -p /etc/snort/rules
sudo mkdir -p /var/log/snort
sudo mkdir -p /etc/snort/rules
sudo mkdir -p /var/log/snort
sudo mkdir -p /usr/local/lib/snort_plugins
sudo mkdir -p /etc/snort/builtin_rules
sudo mkdir -p /etc/snort/so_rules
```

9.1.2 CONFIGURATION:

```
static_networks =
{
    home_net = '10.0.2.0/24',
    external_net = 'any'
}

detection =
{
    global_default_rule_state = true
}

local_rules =
[[
alert icmp any any -> any any (msg:"ICMP packet detected"; sid:1000001;
rev:1;)
alert icmp any any -> any any (msg:"ICMP Ping Sweep";
detection_filter:track by_src, count 5, seconds 1; sid:1000002; rev:1;)
alert tcp any any -> any any (msg:"Potential TCP Port Scan"; flags:S;
detection_filter:track by_src, count 5, seconds 2; sid:1000003; rev:1;)
alert tcp any any -> any 22 (msg:"Potential SSH Brute Force"; flags:S;
flow:stateless; detection_filter:track by_src, count 5, seconds 10;
sid:1000004; rev:1;)
alert tcp any any -> any 80 (msg:"Suspicious User Agent - Possible
Scanner"; flow:established to_server; content:"User-Agent[3A| ";
http_header: pcre:"/(?:Scan|Proxy|Spider|Nmap)/i"; sid:1>
alert tcp any any -> any any (msg:"SQL Injection Attempt";
flow:to_server,established; content:"'%27"; sid:1000006; rev:1;)
alert tcp any any -> any 21 (msg:"FTP Brute Force Attempt";
flow:established to_server; detection_filter:track by_src, count 5, seconds
10; sid:1000007; rev:1;)
]]
```

```

ips =
{
    enable_builtin_rules = true,
    rules = local_rules
}

outputs =
{
    alert_fast =
    {
        enabled = true,
        file = true,
        packet = true
    }
}

include = '/etc/snort/rules/local.rules'

```

9.1.3 PERMISSION SETTING:

```

sudo chmod 666 /var/log/snort
sudo chmod -R 750 /etc/snort
sudo chmod -R 750 /var/log/snort
sudo chmod -R 750 /usr/local/lib/snort_plugins

```

9.1.4 TESTING AND RUNNING:

```

sudo snort -T -c /etc/snort/snort.conf -i eth0
sudo snort -c /etc/snort/snort.conf -i eth0 -v -A alert_fast

```

9.2 APPENDIX 2: OUTPUT SCREENSHOTS

9.2.1 CONFIGURATION SCREENSHOT



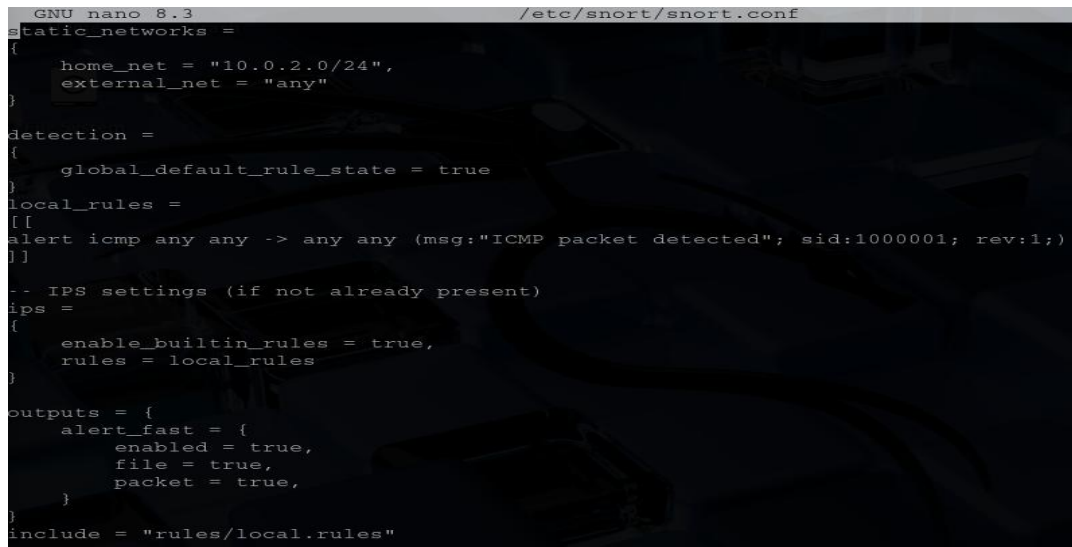
```

(kali㉿kali) - [~]
$ sudo mkdir -p /etc/snort/rules
(kali㉿kali) - [~]
$ sudo mkdir -p /var/log/snort
(kali㉿kali) - [~]
$ sudo mkdir -p /etc/snort/rules
(kali㉿kali) - [~]
$ sudo mkdir -p /var/log/snort
(kali㉿kali) - [~]
$ sudo mkdir -p /usr/local/lib/snort_plugins
(kali㉿kali) - [~]
$ sudo mkdir -p /etc/snort/builtin_rules
(kali㉿kali) - [~]
$ sudo mkdir -p /etc/snort/so_rules
(kali㉿kali) - [~]
$ █

```

Fig 9.2.1.1 – DIRECTORIES CREATION

The above figure shows the command executed in my kali to create the directories.



```
GNU nano 8.3 /etc/snort/snort.conf
static_networks =
{
  home_net = "10.0.2.0/24",
  external_net = "any"
}

detection =
{
  global_default_rule_state = true
}

local_rules =
[[
alert icmp any any -> any any (msg:"ICMP packet detected"; sid:1000001; rev:1;)
]]

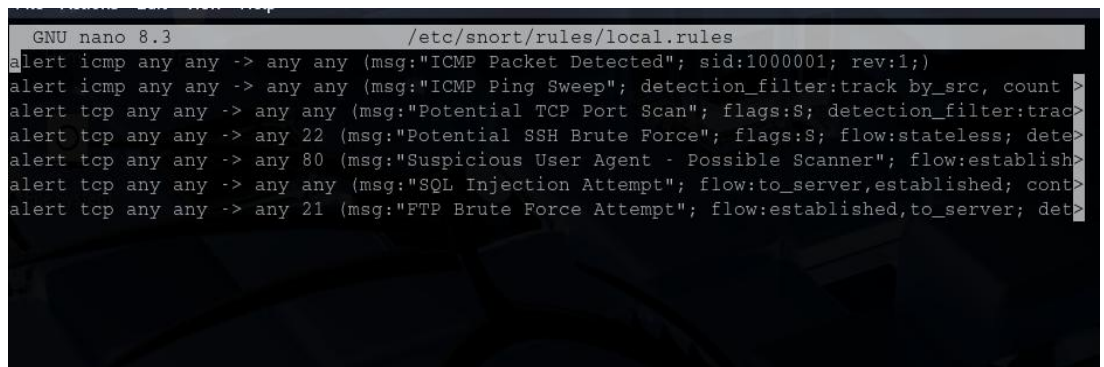
-- IPS settings (if not already present)
ips =
{
  enable_built_in_rules = true,
  rules = local_rules
}

outputs = {
  alert_fast = {
    enabled = true,
    file = true,
    packet = true,
  }
}

include = "rules/local.rules"
```

Fig 9.2.1.2 – SNORT CONFIGURATION

The above figure shows the snort configuration of my system.



```
GNU nano 8.3 /etc/snort/rules/local.rules
alert icmp any any -> any any (msg:"ICMP Packet Detected"; sid:1000001; rev:1;)
alert icmp any any -> any any (msg:"ICMP Ping Sweep"; detection_filter:track by_src, count >
alert tcp any any -> any any (msg:"Potential TCP Port Scan"; flags:S; detection_filter:trac>
alert tcp any any -> any 22 (msg:"Potential SSH Brute Force"; flags:S; flow:stateless; dete>
alert tcp any any -> any 80 (msg:"Suspicious User Agent - Possible Scanner"; flow:establish>
alert tcp any any -> any any (msg:"SQL Injection Attempt"; flow:to_server,established; cont>
alert tcp any any -> any 21 (msg:"FTP Brute Force Attempt"; flow:established,to_server; det>
```

Fig 9.2.1.3 – LOCAL.RULES

The above figure represent the local rules of my network

9.2.2 ALERT LOGS GENERATED

```
Inspection Policy : policy id 0 : /etc/snort/snort.conf
-----
pcap DAQ configured to passive.
-----
host_cache
  memcap: 8388608 bytes
Commencing packet processing
++ [0] eth0
Instance 0 daq pool size: 256
Instance 0 daq batch size: 64
02/23-19:02:24.669574 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} 10
.0.2.15 -> 8.8.8.8
02/23-19:02:24.689899 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} 8.
8.8.8 -> 10.0.2.15
02/23-19:02:24.693907 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} fe
80::2 -> ff02::1
02/23-19:02:24.705451 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} fe
80::f50f:1972:c4f4:85a -> ff02::16
02/23-19:02:24.869596 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} fe
80::f50f:1972:c4f4:85a -> ff02::16
02/23-19:02:25.673803 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} 10
.0.2.15 -> 8.8.8.8
02/23-19:02:25.687925 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} 8.
8.8.8 -> 10.0.2.15
02/23-19:02:26.676360 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} 10
.0.2.15 -> 8.8.8.8
02/23-19:02:26.691497 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} 8.
8.8.8 -> 10.0.2.15
02/23-19:02:27.679587 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} 10
.0.2.15 -> 8.8.8.8
02/23-19:02:27.688732 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} 8.
8.8.8 -> 10.0.2.15
02/23-19:02:28.688324 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} 10
.0.2.15 -> 8.8.8.8
02/23-19:02:28.701900 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} 8.
```

Fig 9.2.2.1 ICMP PACKET DETECT

The above figure represents the ICMP Packet detection alert

```
02/23-20:05:17.104242 [**] [1:1000002:1] "ICMP Ping Sweep" [**] [Priority: 0] {ICMP} 10.0.2.
15 -> 10.2.0.69
02/23-20:05:17.104242 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} 10
.0.2.15 -> 10.2.0.69
02/23-20:05:17.106184 [**] [116:434:1] "(icmp4) ICMP ping Nmap" [**] [Priority: 3] {ICMP} 10
.0.2.15 -> 10.2.0.70
02/23-20:05:17.106184 [**] [1:1000002:1] "ICMP Ping Sweep" [**] [Priority: 0] {ICMP} 10.0.2.
15 -> 10.2.0.70
02/23-20:05:17.106184 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} 10
.0.2.15 -> 10.2.0.70
02/23-20:05:17.110894 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:60335 -> 10.2.0.72:443
02/23-20:05:17.112259 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:60335 -> 10.2.0.73:443
02/23-20:05:17.114528 [**] [116:434:1] "(icmp4) ICMP ping Nmap" [**] [Priority: 3] {ICMP} 10
.0.2.15 -> 10.2.0.74
02/23-20:05:17.114528 [**] [1:1000002:1] "ICMP Ping Sweep" [**] [Priority: 0] {ICMP} 10.0.2.
15 -> 10.2.0.74
02/23-20:05:17.114528 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} 10
.0.2.15 -> 10.2.0.74
02/23-20:05:17.115536 [**] [116:434:1] "(icmp4) ICMP ping Nmap" [**] [Priority: 3] {ICMP} 10
.0.2.15 -> 10.2.0.75
02/23-20:05:17.115536 [**] [1:1000002:1] "ICMP Ping Sweep" [**] [Priority: 0] {ICMP} 10.0.2.
15 -> 10.2.0.75
02/23-20:05:17.115536 [**] [1:1000001:1] "ICMP packet detected" [**] [Priority: 0] {ICMP} 10
.0.2.15 -> 10.2.0.75
02/23-20:05:17.127623 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
```

Fig – 9.2.2.2 PING SWEEP

The above figure represents the ping sweep alerts

```
10.0.2.15:50505 -> 10.0.2.2:1244
02/23-20:09:31.976109 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:3351
02/23-20:09:32.076550 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:3261
02/23-20:09:32.177766 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:5102
02/23-20:09:32.277889 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:40911
02/23-20:09:32.379051 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:1022
02/23-20:09:32.479828 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50514 -> 10.0.2.2:2702
02/23-20:09:32.579954 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:31337
02/23-20:09:32.681359 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:2222
02/23-20:09:32.781920 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:5961
02/23-20:09:32.882227 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:2382
02/23-20:09:32.982686 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:6901
02/23-20:09:33.082819 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:16012
02/23-20:09:33.183182 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:1761
02/23-20:09:33.190446 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50512 -> 10.0.2.3:53
02/23-20:09:33.283546 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:5815
02/23-20:09:33.384108 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:5225
02/23-20:09:33.484533 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:1113
02/23-20:09:33.584604 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
10.0.2.15:50505 -> 10.0.2.2:9090
02/23-20:09:33.685528 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
```

Fig – 9.2.2.3 PORT SCAN

The above figure represents the port scans

```
10.0.2.2 -> 10.0.2.15
02/23-20:20:39.410303 [**] [116:409:1] "(ipv4) IPv4 packet to 'current net' dest address" [*]
*] [Priority: 3] {TCP} 10.0.2.15:36260 -> 0.0.0.22:3306
02/23-20:20:39.410936 [**] [116:409:1] "(ipv4) IPv4 packet to 'current net' dest address" [*]
*] [Priority: 3] {TCP} 10.0.2.15:36260 -> 0.0.0.22:113
02/23-20:20:39.411053 [**] [116:408:1] "(ipv4) IPv4 packet from 'current net' source address"
*] [**] [Priority: 3] {TCP} 0.0.0.22:3306 -> 10.0.2.15:36260
02/23-20:20:39.411034 [**] [116:409:1] "(ipv4) IPv4 packet to 'current net' dest address" [*]
*] [Priority: 3] {TCP} 10.0.2.15:36260 -> 0.0.0.22:8888
02/23-20:20:39.411067 [**] [116:409:1] "(ipv4) IPv4 packet to 'current net' dest address" [*]
*] [Priority: 3] {TCP} 10.0.2.15:36260 -> 0.0.0.22:5900
02/23-20:20:39.411112 [**] [116:409:1] "(ipv4) IPv4 packet to 'current net' dest address" [*]
*] [Priority: 3] {TCP} 10.0.2.15:36260 -> 0.0.0.22:25
02/23-20:20:39.411146 [**] [116:409:1] "(ipv4) IPv4 packet to 'current net' dest address" [*]
*] [Priority: 3] {TCP} 10.0.2.15:36260 -> 0.0.0.22:445
02/23-20:20:39.411146 [**] [1:1000003:1] "Potential TCP Port Scan" [**] [Priority: 0] {TCP}
```

FIG – 9.2.2.4 SSH

The above figure represents the ssh scan